Custom Binary Analysis Tool

July 16, 2025

Overview

- Automate reverse engineering tasks for ELF binaries.
- Structure code as:
  - **Go**: Packages with exported functions.
  - **Python**: Modules with clear function structure.
  - **Bash**: Standalone utility functions, including a `main` orchestrator.
- Accept necessary filenames, options, or arguments as input parameters.
- Deliver clear usage documentation and, if possible, sample binaries for demonstration.

Module Topics (Choose Two or More)

1. **File Information Fetch**: Extract metadata like file type, size, ELF class, permissions, and security flags (RELRO, PIE, NX).
2. **System Call Analysis**: Identify and enumerate system calls the binary uses via static or dynamic analysis.
3. **ROP Gadgets and Return Address Discovery**: Locate return addresses and potential ROP (Return-Oriented Programming) gadgets within the binary.
4. **Symbol Table Extraction**: List functions, variable names, and segment data from the binary's symbol table.
5. **Function Flow/Graph Parsing**: Parse assembly/disassembly outputs to produce function call graphs or simplified logic workflows.
6. **Other Reverse Engineering Insights**: Any extended or creative analysis such as CFG analysis, entry point tracing, static/dynamic diffing etc.

Example: Sample Function Structures

Python (as module)

```
def fetch_file_info(filename: str) -> dict:
    """
    Extracts ELF metadata and security flags from the file.
    Args:
        filename: path to ELF file.
    Returns:
        Dictionary with file info and protection status.
```

```
    """

def extract_syscalls(filename: str) -> list:
    """
    Identifies all system calls referenced in ELF binary.
    """
```

Go (as package)

```
// Package reverseeng provides ELF analysis utilities.
package reverseeng

func FetchFileInfo(filename string) (FileInfo, error) {
    // Implementation
}

func ExtractSyscalls(filename string) ([]string, error) {
    // Implementation
}
```

Bash (as script with functions)

```
fetch_file_info() {
    local filename="$1"
    # Use 'readelf' or 'file' commands
}
main() {
    fetch_file_info "$1"
}
main "$@"
```

Recommended Implementation Details

- Use robust parsing (e.g., Python's `pyelftools`, Bash's `readelf`, Go's `debug/elf`).
- Accept file names as arguments; fail gracefully on invalid input.
- Output results in readable or parsable format (JSON, table, plaintext).
- Where possible, write test cases (e.g., unit tests for Python/Go, sample script input in Bash).
- Document edge cases, limitations, and additional findings.

Example Things You Can Do in Each Section

Section	Potential Approaches/Tasks
File Info Fetch	File size, ELF class (32/64 bit), endianness, section headers, security bits (RELRO, PIE, NX), symbol/version check, printing readable report.
System Call Analysis	List all syscall numbers, map syscall names, identify sensitive syscalls (execve, open, ptrace), analyze dynamic vs static syscalls.
ROP	Find ROP gadgets (using e.g. `ROPgadget`, `radare2`), print return addresses, enumerate short instruction sequences ending in 'ret'.
Symbol Table	List exported symbols, imported symbols, external calls, addresses, types (function, object, section).
Call Graph / Workflow	Produce a call graph from assembly (dot/graphviz output), or simplified function workflow, detect recursive calls, function entry/exit points.
Others	Pattern matching for obfuscation, entropy analysis, CFG testing, function boundary recovery.

Submission Guidelines

- Submit at least one or two modules (more are welcome).
- Include:
  - Well-commented code.
  - Clear documentation and usage instructions.
  - Sample outputs or screenshots.
  - (Optional) Test cases and/or sample binaries.
- Ensure code is modular and reusable for future reverse engineering work.

Evaluation Criteria

- **Correctness**: Functional and accurate extraction or automation.
- **Code Quality**: Readable, well-structured, idiomatic for language.
- **Modularity & Reusability**: Functions/components decoupled as needed.
- **Documentation**: Comprehensive explanation and use-cases.
- **Extension/Creativity**: Novel insights or techniques beyond basic requirements.

References and Tips

- Use available reverse engineering libraries and tools (e.g. `pyelftools`, `readelf`, `objdump`, `radare2`).
- Refer to the resource mentioned in the learning modules repo.

Why Do This ??

Once you submit your task, we will review and test each of your module implementations. Well-written, correct, and modular code will be merged into a one full reverse engineering toolkit to be used internally during and after this tenure